

ОС Материал Семинаров\

Page • Tag

Про регистры

Регистры общего назначения (x86)

В реальном режиме:

AX — аккумулятор

BX — базовый регистр

CX — счётчик

DX — регистр данных

Сегментные регистры

CS — сегмент кода

DS — сегмент данных

SS — сегмент стека

ES — дополнительный сегмент

FS, GS — дополнительные сегменты

IP (EIP / RIP) — указатель текущей инструкции

FLAGS (EFLAGS / RFLAGS) — флаги состояния процессора

Управляющие регистры (защищённый режим)

CR0 — состояние процессора
(включение защищённого режима, страничной адресации и т.д.)

CR1 — зарезервирован

CR2 — адрес страницы при ошибке (page fault)

CR3 — база таблиц страниц

в long mode: адрес таблицы страниц 4-го уровня

CR4 — дополнительные возможности
(включение PAE, SSE и др.)

Регистры базового адреса (служебные)

GDTR — регистр таблицы глобальных дескрипторов (GDT)

IDTR — регистр таблицы прерываний (IDT)

LDTR — регистр локальной таблицы дескрипторов (LDT)

Компьютер — программно-управляемое устройство. Процессором управляют нанопрограммы.

Функции, которые INT 8h

1. Инкремент счётчика реального времени по определённому адресу.
Повторяется с частотой 18.2 тика в секунду.



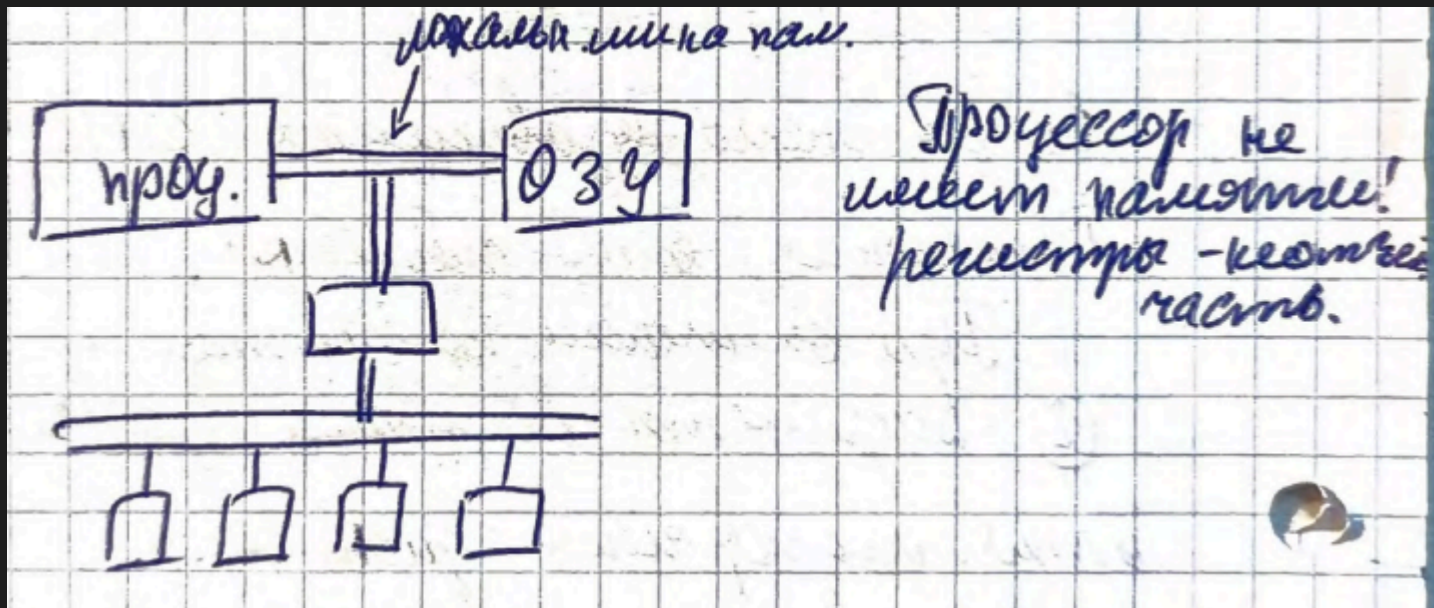
2. Декремент счётчика времени до отключения моторчика дисководов.
Все запоминающие устройства — это внешние устройства.
Выключение моторчика дисководов — это отложенное действие.
Посылается команда на порт дисководов.

3. Вызов пользовательского прерывания 1Ch

Префиксная команда LOCK делает следующую команду неделимой. Выполнение данной команды нельзя прервать.

Аппаратно: блокировка локальной шины памяти на время выполнения этой команды.

Процессор соединен с ОЗУ через локальную шину памяти



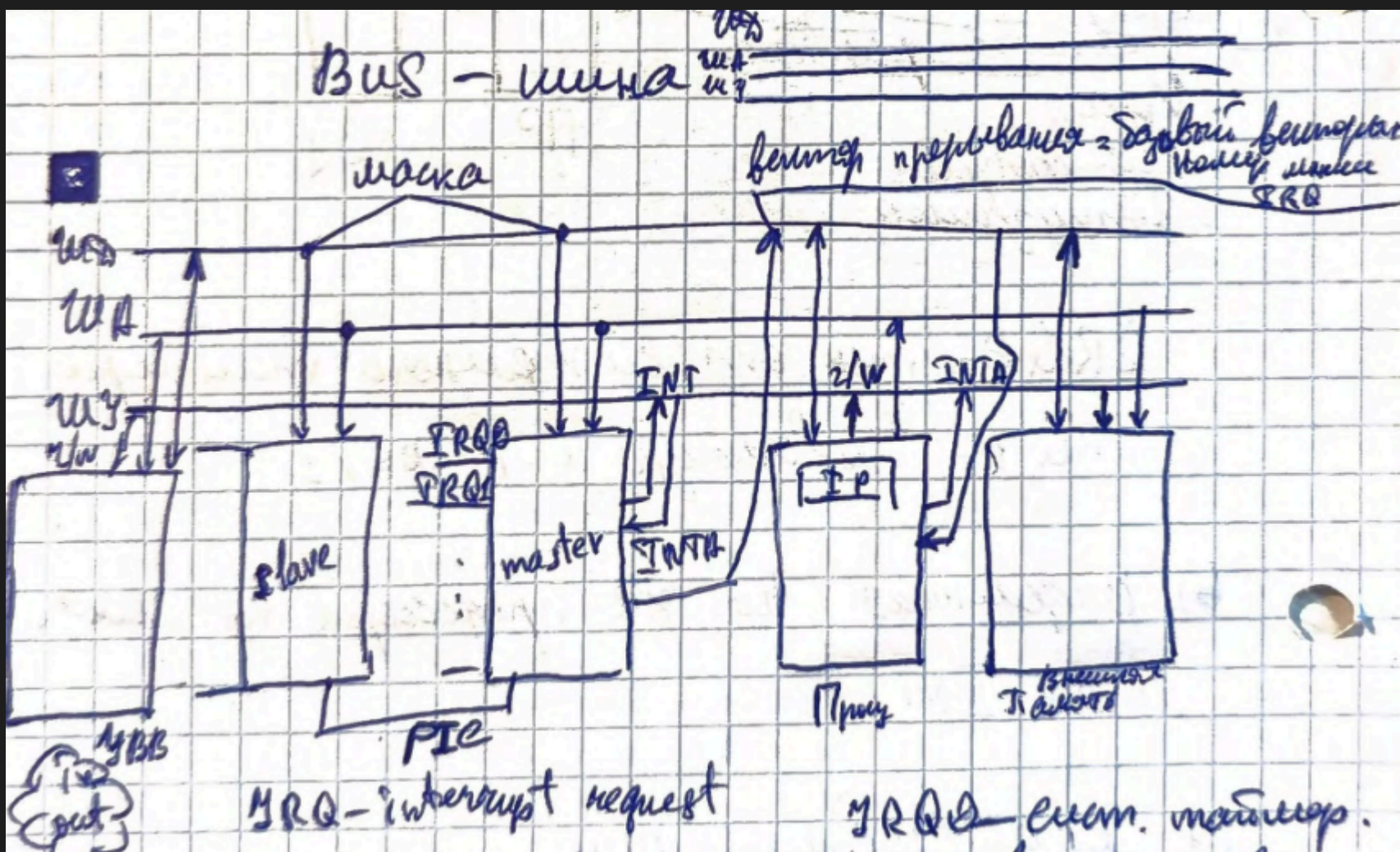
Байт — минимальная адресная единица памяти.

Сигналы трёх типов:

1. Данные: собственно данные и команды.
2. Адреса
3. Сигналы управления

Поэтому в концептуальной схеме компьютера три отдельные шины:

- данных
- адреса
- управления



IRQ0 — тик от системного таймера

IRQ1 — клавиатура и в PS/2

ОС — это комплекс аппаратных, программных средств, комплекс (совместно используемых) управляющих ресурсов вычислительной системы и процессов, которые используют эти ресурсы.

Процесс — основа декомпозиции UNIX. Владелец ресурсов является процессом.

В UNIX процесс создаётся вызовом `fork()`. `fork` → `clone`. Может создать процесс или поток.

Иерархия процессов. Процесс с id 0 и 1.

0 — swapper (запуск системы)

1 — init

PID — process identifier

PPID — parent process id

PGID — process group id

kthreadd — kernel thread daemon — создаёт потоки ядра.

Plain Text ▾

PROCESS FLAGS top

The sum of these values is displayed in the "F" column, which is provided by the flags output specifier:

1 forked but didn't exec
4 used super-user privileges

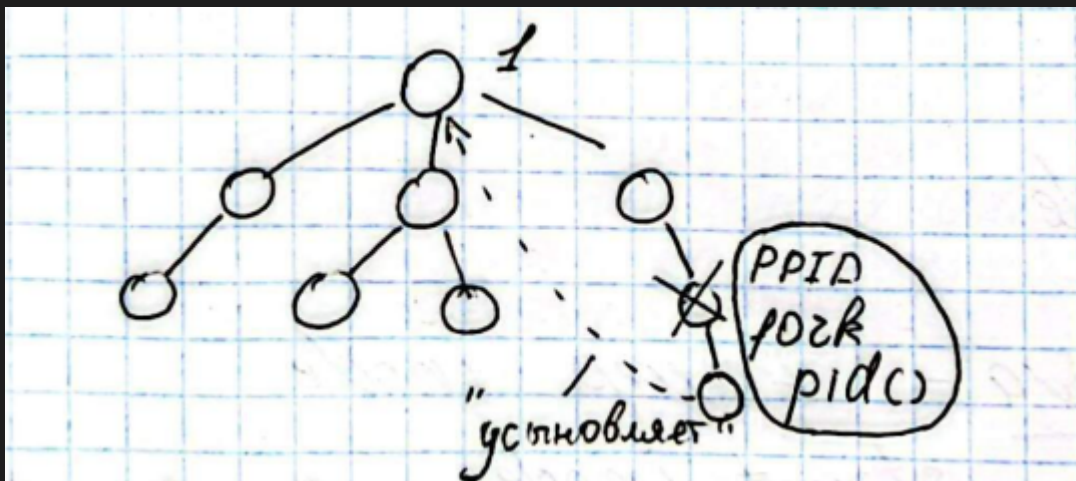
PROCESS STATE CODES top

Here are the different values that the s, stat and state output specifiers (header "STAT" or "S") will display to describe the state of a process:

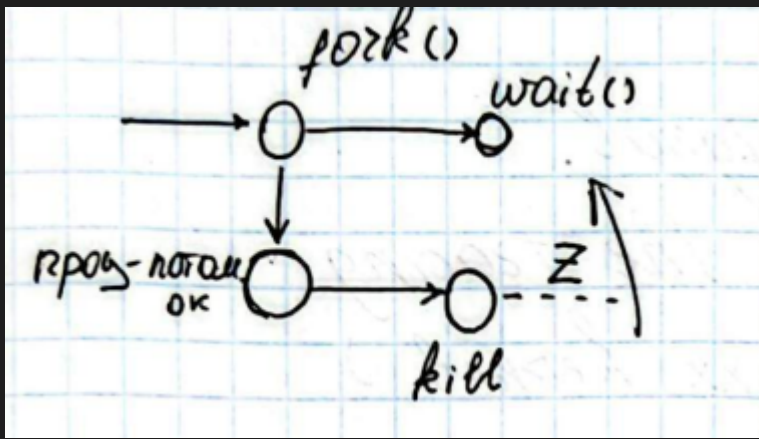
D uninterruptible sleep (usually I/O)
I idle kernel thread
R running or runnable (on run queue)
S interruptible sleep (waiting for an event to complete)
T stopped by job control signal
t stopped by debugger during the tracing
W paging (not valid since Linux 2.6)
X dead (should never be seen)
Z defunct ("zombie") process, terminated but not reaped by its parent

For BSD formats and when the stat keyword is used, additional characters may be displayed:

< high-priority (not nice to other users)
N low-priority (nice to other users)
L has pages locked into memory (for real-time and custom I/O)
s is a session leader
l is multi-threaded (using CLONE_THREAD, like NPTL pthreads do)
+ is in the foreground process group



wait() — ожидание процессом-предком завершения процесса-потомка и получение статуса завершения потомка.



Процесс Зомби и процесс Сирота

Если завершается процесс-предок, возникает процесс-сирота, и его усыновляет процесс, открывший терминал.

Функция `wait()` предназначена для ожидания завершения процесса-потомка и получения его статуса завершения.

После вызова `fork()` у родительского процесса вызывается `wait()`, он блокируется, пока процесс-потомок не завершится, после чего родитель получает статус завершения потомка. Процесс-потомок при нормальном завершении возвращает управление системе, а при принудительном завершении может быть завершён сигналом.

Процесс «зомби» — это процесс, у которого отобраны все ресурсы, кроме последнего, а именно дескриптора процесса, то есть записи, содержащей информацию о завершении.

В системе нет таблицы процессов в виде массива. Вместо этого используются связанные списки процессов.

Таблица в классическом смысле — это массив структур. Работа с таким массивом требует сдвига элементов при вставке и удалении, что приводит к большим накладным расходам.

Процесс-зомби существует до тех пор, пока процесс-предок не вызовет функцию `wait()` или пока сам процесс-предок не завершится.

Файлы.

`ls -al` `ls -ial`.

Столбик с i-node

Идентификатор — номер i-node

Дескриптор — `struct i-node`

hard link — один и тот же файл

soft link — специальный файл. Внутри файла строка с путём к файлу.

Типы файлов (символ в начале `ls -l`):

- `-` — обычный файл (regular file)
- `d` — директория (directory)
- `l` — символическая ссылка (symbolic link)
- `b` — блочное устройство (block device)
- `c` — символьное устройство (character device)
- `p` — именованный канал (FIFO / pipe)
- `s` — сокет (socket)

Формат `ls -l` (пример):

```
drwxr-xr-x  2 owner group 4096 2026-01-14 name
```

первый столбец: тип + 9 бит прав; далее — счётчик ссылок, владелец, группа, размер, время, имя. `-i` добавляет номер i-node.

Права доступа: трёхкратная группа битов `rwX rwX rwX`:

- первая тройка — владелец (user)
- вторая — группа (group)
- третья — остальные (others)

Значение битов:

- `r` — право чтения

- `w` — право записи (изменение содержимого; в директории — создание/удаление/переименование записей)
- `x` — для файла: выполнение; для директории: доступ по имени

Системные вызовы `fork` и `exec`

Защищённая виртуальная адресация:

ни один процесс не может обращаться в адресное пространство другого процесса.

наследуется от процесса-предка:

код исполняемой программы, сигнальные маски, маску режима создания файлов,

`fork()` — системный вызов, который создаёт новый процесс — процесс-потомок, для которого создаётся собственное адресное пространство, т. е. создаются собственные карты трансляции АДРЕСОВ, дескрипторы СТРАНИЦ ТАБЛИЦ СТРАНИЦ потомка ссылаются на СТРАНИЦЫ адресного пространства предка И для страниц адресного пространства предка права доступа МЕНЯЮТСЯ на `only-read` и УСТАНОВЛИВАЕТСЯ флаг `copy-on-write`

И если или предок, или потомок попытаются изменить какую-либо страницу, возникнет исключение по правам доступа И операционная система, обрабатывая это исключение, ОБНАРУЖИТ флаг `copy-on-write` и создаст копию страницы в адресном пространстве того процесса, который пытался её изменить

И такая ситуация существует до тех пор, пока процесс-потомок не вызовет `exec()` или `exit()` (тут именно `exit()`, а не завершение процесса) И в результате страницам адресного пространства предка возвращаются обычные права доступа и флаг `copy-on-write` сбрасывается.

Что такое `exec()`?

`exec()` — системный вызов, который переводит процесс на адресное пространство программы, которая была передана ему в качестве параметров

`exec` выполняет следующие действия:

1. Разбирает путь к исполняемому файлу, чтобы проверить существует ли этот файл.
2. Проверяет, имеет ли вызывающий процесс полномочия на выполнение файла
3. Читает заголовки и проверяет, что он действительно исполняемый.
4. Освобождение занимаемых процессом ресурсов (+ области своппинга)
5. Создаются адресное пространство новое для программы, переданной в качестве параметра, то есть создаются новые таблицы страниц.
6. В дескрипторе процесса записывается адрес новой таблицы страниц, в регистр `cr3` записывается адрес новой таблицы страниц
7. После того, как в дескрипторе заменен адрес, для начала выполнения программы необходимо сбросить значения регистров процессора, в счетчик команд записывается адрес точки входа.
8. Программа начинает выполняться

Взаимодействие между процессами.

В многопроцессной ОС процессы могут взаимодействовать друг с другом.

Одно из таких средств — программные каналы (`pipe` = труба).

Потоковая модель передачи данных.

В каналах осуществляется симплексная связь.

Неименованные программные каналы создаются системным вызовом `pipe()`;

В неименованных каналов сть дескриптор `inode`.

Именованный программный канал — это файл.

Программные каналы в Unix поддерживаются файловой системой.

Программные каналы именованные и неименованные — буферы, создаваемые в области данных ядра ОС.

Размер установлен в системе: 4096 байт.

Пользователь не указывает размер буфера.

Адресные пространства процессов защищённые ни один процесс не может обратиться в адресное пространство другого процесса ⇒ взаимодействовать, т.е. передавать данные можно через третье адресное пространство — адресное

пространство ядра системы.

Прочитанные сообщения перестают существовать.

IPC System V. Сигналы. Signals

В стандарт **IPC System V** входят следующие средства:

Сигналы.

Семафоры.

Программные каналы: неименованные и именованные.

Сегменты разделяемой памяти.

Очереди сообщений.

Файлы, отображаемые в память (в курсе не рассматриваются).

Сигналы

- Важнейшее средство информирования в UNIX о событиях, происходящих вне процессов и в самих процессах.
- Способ информирования процесса ядром о происшествии какого-то события.

Иерархия и группы процессов в контексте сигналов:

Зачем поддерживается иерархия? Статусы завершения получают постепенно предки по иерархии. Они не приходят в систему все сразу. Предок может по-разному реагировать на статус завершения. Это делает систему более гибкой.

Поддержка группы процессов: Процессы одной группы получают одни и те же сигналы (при посылке сигнала группе).

Реакция процесса на сигнал

Процесс может реагировать на сигнал тремя способами:

Игнорировать сигнал.

Реагировать на сигнал **способом по умолчанию** (SIG_DFL), определённым в системе.

Определить **свою реакцию** (перехватить сигнал) путём регистрации собственных обработчиков сигналов в системе.

#define NSIG 20

SIGHUP (1) — Сообщает процессу об утрате связи с терминалом (используется в демонах).

SIGINT (2) — Прерывание выполнения процесса (генерируется Ctrl+C).

SIGQUIT (3) — Аварийное завершение (Ctrl+\\).

SIGKILL (9) — Безусловное уничтожение процесса (нельзя перехватить или игнорировать).

SIGBUS (10) — Ошибка шины (недопустимое обращение к памяти).

SIGSEGV (11) — Нарушение защиты памяти (выход процесса за пределы его адресного пространства).

SIGPIPE (13) — Запись в канал, когда нет читателя (запись есть, read нет).

SIGALRM (14) — Сигнал от таймера (будильник).

SIGTERM (15) — Сигнал завершения (мягкое, "вежливое" завершение).

SIGUSR1 (16), SIGUSR2 (17) — Пользовательские сигналы.

SIGCHLD (18) — Сигнал от потомка к процессу-родителю (при изменении статуса потомка).

SIGPWR (19) — Сбой/ошибка питания.

SIGPOLL (22) — Событие в стрим-устройстве (ввод-вывод).

SIG_DFL — (void (*)(int)) 0 — действие по умолчанию.

SIG_IGN — (void (*)(int)) 1 — игнорировать сигнал.

Средство отправки и восприятия сигналов реализуется через 2 системных вызова: kill и signal.

A) Системный вызов kill

Отправка сигнала процессу или группе.

codeC

Plain Text ▾

```
int kill(int pid, int sig);
```

pid: идентификатор процесса-получателя.

Если pid ≤ -1: сигнал отправляется **группе** процессов (всем родственным процессам).

Пример (отправка самому себе): kill(getpid(), SIGALRM);

Б) Системный вызов signal

Регистрация обработчика сигнала.

Прототип (через typedef):

codeC

Plain Text ▾

```
#include <signal.h>
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
```

Прототип (полный):

codeC

Plain Text ▾

```
void (*signal(int signum, void (*handler)(int)))(int);
```

handler — это обработчик сигнала (функция).

signum — номер сигнала.

Функция signal возвращает указатель на **предыдущий** обработчик сигнала (чтобы его можно было восстановить).

Пример использования (с восстановлением старого обработчика):

codeC

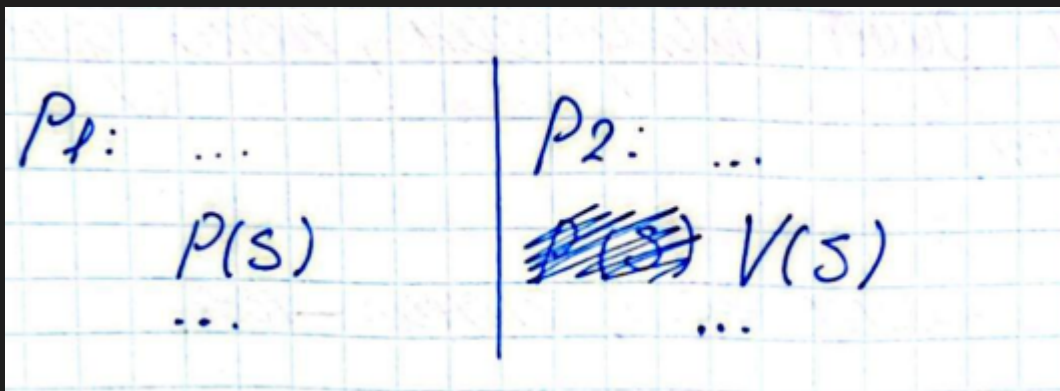
Plain Text ▾

```
int main() {
    // Устанавливаем свой обработчик и сохраняем старый
    void (*old_handler)(int) = signal(SIGINT, my_handler);

    // ... критический участок кода ...

    // Восстанавливаем старый обработчик
    signal(SIGINT, old_handler);
}
```

Семафоры



P1 будет ждать (блокироваться), пока P2 не освободит семафор V(s)

Наборы семафоров:

Все современные ОС поддерживают **наборы** (массивы) семафоров.

Основная особенность: Одной неделимой (атомарной) операцией можно изменить состояние **всех** или **части** семафоров набора.

Семантически это массивы семафоров.

Семафоры поддерживаются **таблицей семафоров в ядре** системы. Таблица — это массив структур.

Структура semid_ds (<sys/sem.h>) содержит информацию о наборе:

Имя (key): Целое число, присваивается процессом, создавшим набор. Другие процессы могут открыть набор по этому имени и получить его дескриптор.

UID: Идентификатор пользователя-создателя. Процесс с совпадающим UID может изменять набор (например, удалять или менять права доступа).

Права доступа: Стандартные права UNIX для трех групп (user/group/others — read/write/execute).

Количество семафоров в наборе.

Время изменения одного или нескольких значений семафоров.

Время последнего изменения управляющих параметров набора.

Указатель на сам массив семафоров.

Системные вызовы

Для работы с семафорами используются три основных вызова:

semget() — создает набор семафоров (или получает доступ к существующему).

semctl() — управление набором (установка значений параметров, удаление).

semop() — операции над семафорами.

A) Функция semget

codeC

Plain Text ▾

```
int semget(key_t key, int nsems, int semflg);
```

key — идентификатор (ключ).

nsems — количество семафоров в наборе.

semflg — флаги (например, IPC_CREAT | perms).

Пример получения ключа и создания:

Plain Text ▾

```
key_t semkey = ftok("./key.txt", 1);
if (semkey == -1) {
    perror("ftok");
    exit(1);
}
```

Б) Функция semop

Plain Text ▾

```
int semop(int semfd, struct sembuf *op_ptr, unsigned int len);
```

semfd — дескриптор набора.

op_ptr — указатель на массив структур sembuf.

len — размер массива структур (количество операций).

Структура sembuf:

Plain Text ▾

```
struct sembuf {
    ushort sem_num; // Индекс семафора в наборе (начинается с 0)
    short  sem_op;  // Операция
    short  sem_flg; // Флаги
};
```

В System V определено 3 типа операций

- **sem_op < 0 (Захват/Декремент)**
- **sem_op = 0 (Проверка на ноль без захвата)**
- **sem_op > 0 (Освобождение/Инкремент)**

IPC_NOWAIT — сообщает операционной системе о нежелании процесса блокироваться.

Если процесс заблокирован, он не получает процессорного времени.

Это сделано для того, чтобы процессы, стоящие в очереди к семафору, не были заблокированы при аварийном завершении процесса, захватившего семафор.

Захват семафора — критическая операция.

SEM_UNDO — сообщает системе, что она должна отслеживать изменения семафора в результате sem_op и в случае завершения процесса отменять их.

C++ ▾

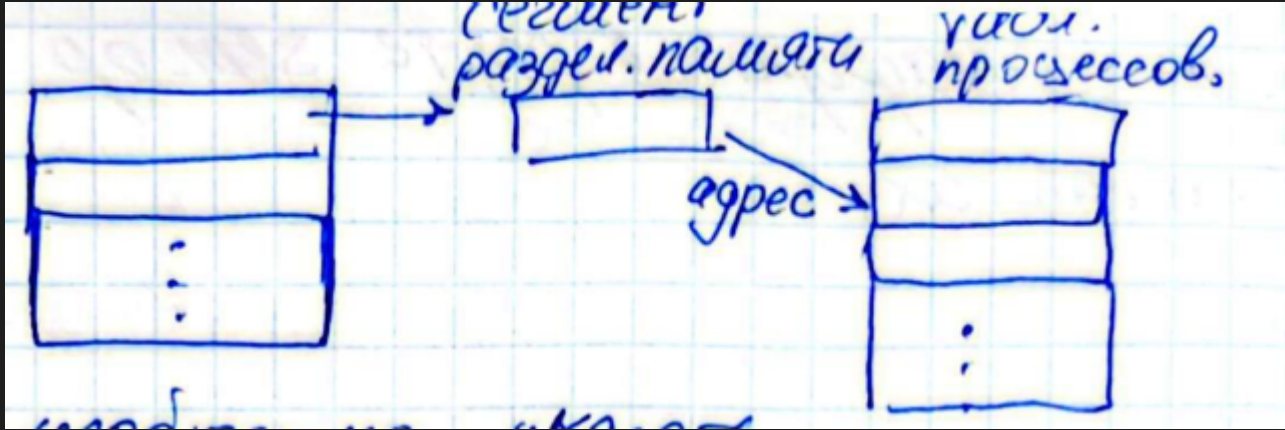
```
#include <sys/sem.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <stdlib.h>
struct sembuf sops[2] = {
    {0, -1, SEM_UNDO | IPC_NOWAIT},
    {1, 0, 1}
};
int main() {
    int perms = S_IRWXU | S_IRWXG | S_IRWXO;
    int fd = semget(100, 2, IPC_CREAT | perms);
    if (fd == -1) {
        perror("semget"); exit(1);
    }
    if (semop(fd, sops, 2) == -1) {
        perror("semop"); exit(1);
    }
    return 0;
}
```

Разделяемая память (Shared memory)

На ней нет средств взаимного исключения. Используется совместно с семафорами. В отличие от pipe, где сообщения исчезают после прочтения (потокковая модель) — это простой буфер. Это простой участок, который можно записать и многократно читать.

Таблица разделяемой памяти находится в ядре.

```
#include <sys/shm.h>
struct shmid_ds
```



Для разделяемых сегментов необходимо указать размер.

Системные вызовы:

shmget() — создание сегмента;

shmctl() — управление;

shmat() — attach (присоединить);

shmdt() — detach (отсоединить).

```
void shmat (int shmid, const void shmaddr, int shmflg);
```

shmaddr — предпочтительный адрес, куда сделать attach. Если 0 → может быть установлен любой подходящий адрес (система определяет сама).

C++ ▾

```
#include <sys/shm.h>
#include <sys/types.h>
#include <sys/ipc.h>
int main() {
    int perms = S_IRWXU | S_IRWXG | S_IRWXO;
    int fd = shmget(100, 1024, IPC_CREAT | perms);
    if (fd == -1) { perror("shmget"); exit(1); }

    char *addr = (char *) shmat(fd, 0, 0);
    if (addr == (char *) -1) { perror("shmat"); exit(1); }

    strcpy(addr, "abcd");

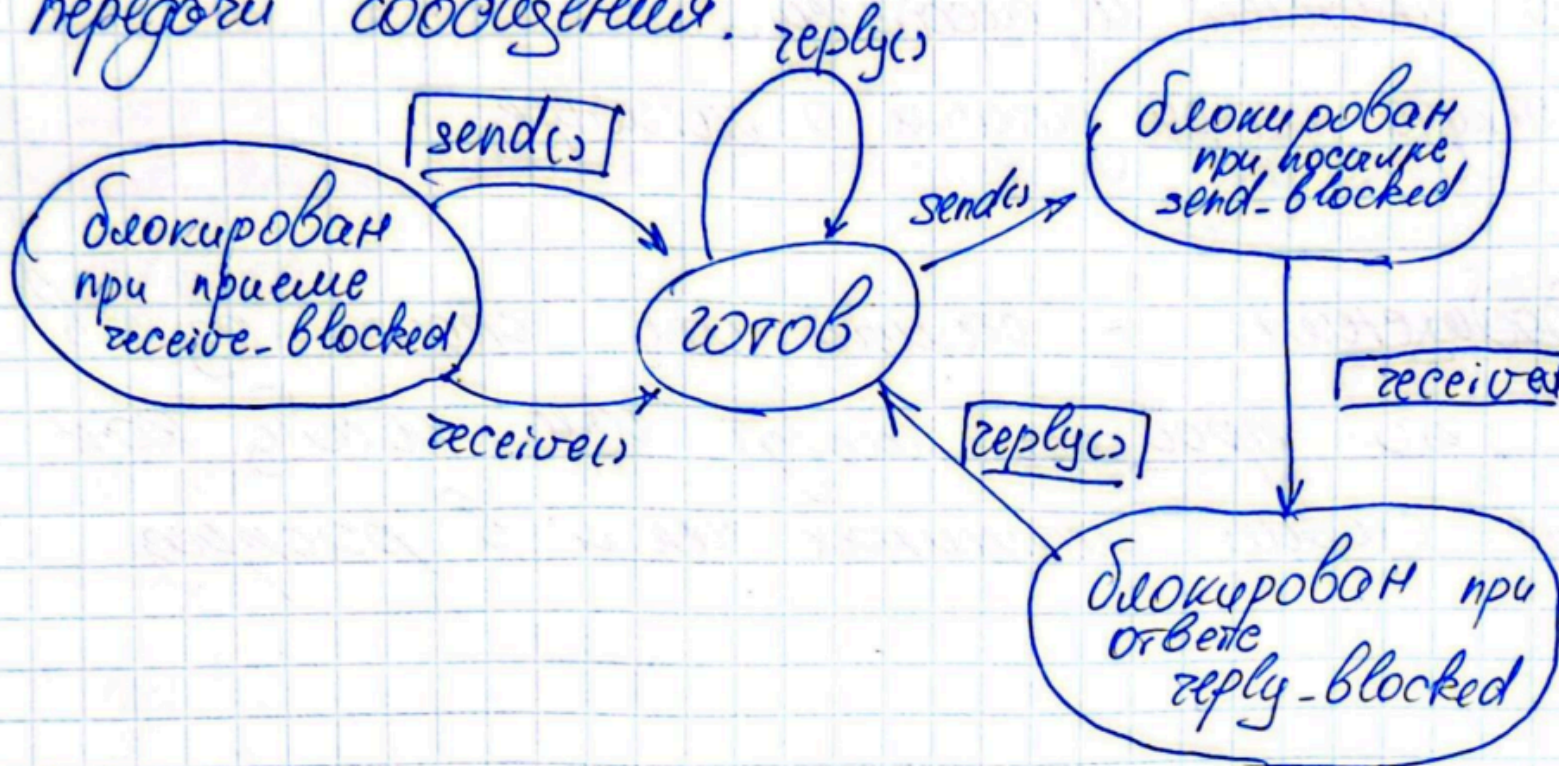
    if (shmdt(addr) == -1) { perror("shmdt"); }
    return 0;
}
```

Рассмотрим 2 пр
 P1 заинтересован в том, чтобы P2 выполнил для
 него работу.



Синхронизация ≠ взаимное исключение.
 Читатели ждут, если никто не пишет.

В Диаграмма 3 сост. блокировка процесса при
 передаче сообщения. get()



Потерянное обн. нужно, чтобы читатели получали актуальную информацию!

От receive-blocked стрелка reply к Блокирован при ответе reply-blocked.

Очереди сообщ. поддерж. сис. таблицей очередей сообщений.

```

<sys/msg.h>
struct msgid_ds

```

msg_spot - указатель на обл. данных ядра, в кот. находится сообщение.

```

msgget();
msgctl();
msgsnd();
msgrcv();

struct msgbuf {
    long msgtype;

```

```
char msgtext[MSGMAX];
}
```

Когда процесс передаёт сообщение в очередь, ядро создаёт для него новую запись и помещает её в конец связанного списка записей очереди.

В каждой записи указывается тип, число байт данных, содержащихся в сообщении, и указатель на область данных ядра, где фактически находится содержимое сообщения.

Ядро копирует содержимое сообщения из области данных процесса-отправителя в область данных ядра, чтобы процесс-отправитель мог завершиться, а сообщение оставалось доступным для других процессов.

Когда процесс извлекает сообщение из очереди, ядро копирует содержимое сообщения в адресное пространство процесса, после чего оно перестает существовать.

Процесс может выбрать сообщ. из очереди 3-мя сп:

1. Самое старое вне зав-ти от типа (в начале очереди)
2. Если его инентификатор совп. с тем, что указал процесс. (выбор по инентификатору). Если таких сообщ. $\exists$ несколько, то будет взято самое старое.
3. Тип которого наименьш. из меньших или равно тому, что указал процесс. Если таких несколько - то самое старое.

Сравнение с разделяемой памятью:

В отличие от разделяемых сегментов, при работе с которыми исключено копирование данных, здесь их **два**:

Из адресного пространства отправителя в ядро.

Из ядра в адресное пространство получателя.

Но в отличие от [других средств IPC], могут быть исключены блокировки (с помощью флага IPC_NOWAIT).

C++ ▾

```
#include <sys/msg.h>
#include <sys/ipc.h>
#include <sys/types.h>
#include <string.h>
#include <stdio.h>
#ifndef MSGMAX
#define MSGMAX 1024
#endif
struct mbuf {
    long mtype;           // Тип сообщения
    char mtext[MSGMAX];   // Текст сообщения
} mobj = { 15, "HELLO" }; // Инициализация: тип 15, текст "HELLO"

int main() {
    int fd = msgget(100, IPC_CREAT | IPC_EXCL | 0624);
    if (fd != -msgsnd(fd, &mobj, strlen(mobj.mtext) + 1, IPC_NOWAIT))
        perror("msg");
    return 0;
}
```

После отправки процесс может завершиться, и сообщение **остается доступным** для других процессов (так как оно хранится в ядре).

Блокировки могут быть исключены (с использованием флага IPC_NOWAIT)

Важно: Нельзя оценить, сколько времени процесс проведет в блокировке!

Чтобы создать процесс-демон, необходимо выполнить следующую последовательность действий:

1. **Сбросить маску режима создания файлов:** вызвать `umask(0)`. Это нужно, чтобы демон имел полные права на создаваемые им файлы.

2. Вызвать системный вызов `fork` и завершить родительский процесс.
3. Создать новую сессию через `setsid`. (Процесс становится лидером новой сессии и новой группы.)
4. Сделать корневой каталог текущим рабочим каталогом
Это необходимо, чтобы демон не удерживал файловую систему (например, если он запущен с флешки, её нельзя будет отмонтировать)
5. Закрывать все ненужные файловые дескрипторы.
6. Опционально: Открыть файловые дескрипторы с номерами 0, 1 и 2 (`stdin`, `stdout`, `stderr`) на устройстве `/dev/null`.

Root нужен для портов < 1024 и записи PID-файла в `/var/run`

Планирование и Время

Тик (jiffies): Выделенный период таймера (не частота).

Диспетчеризация: Обработчик таймера декрементирует квант. Квант = 0

передача управления.

Приоритеты:

Процессы уровня пользователя — **динамические**.

Потоки ядра и Real-time — **статические**.

Главный тик: Период (несколько обычных тиков), по которому происходит пересчет приоритетов и запуск своппера.

Управление памятью

Page Fault: Обращение к странице, отсутствующей в памяти (Frame). Единственное исправимое исключение.

Своппинг:

Задача `page-daemon` (`swapper`) — выгружать страницы заранее, а не в момент нехватки.

Действия своппера и планировщика инициируются как **отложенные действия** (аналог DPC в Windows), чтобы не вешать обработчик прерывания с высоким приоритетом.

Системы разделения времени

Системы разделения времени появились как альтернатива системам пакетной обработки. Появились вместе с первыми мониторами. В результате у обычных пользователей появилась возможность, сидя за терминалами, удаленно соединенными телефонным кабелем, взаимодействовать с программным обеспечением. Это потребовало переписать операционные системы, и они стали системами разделения времени. Система, в которой время разделяется между пользователями. Такая система должна обеспечивать гарантированное время отклика. Time-slicing системы.

Квантование и диспетчеризация

Квант — отрезок времени определенной длины. По истечении кванта основная задача системы — передать время другому процессу или потоку, то есть диспетчеризация. Для единообразия обработки: если в процессе нет дополнительных потоков, то он рассматривается как основной поток.

Системный таймер

Тик — это не тактовая частота, а выделенный период. Основная функция обработчика прерывания — декремент кванта. Когда счетчик кванта достигает нулевого значения.

Планирование и приоритеты

Приоритетное планирование с вытеснением. При этом приоритеты процессов уровня пользователя пересчитываются. Процессы реального времени и потоки ядра имеют статический приоритет. Задача пересчета приоритетов — планировщик. Когда истек квант, пересчитывать приоритет поздно. Главный тик — период, равный нескольким тикам. В главный тик происходит пересчет приоритетов.

Управление памятью

Иницируется работа своппера по главному тiku. Процессы имеют только виртуальное адресное пространство, а в физическую память загружается страница, к которой процессор обращается в текущий момент времени. Если процесс обращается к странице, отсутствующей в физической памяти, возникает страничное прерывание. Это страничное прерывание относится к исключениям и это единственное исправимое исключение в системе. Страница — это страница виртуального адресного пространства. Frame — страница физической памяти. Выгружать страницу, когда уже загружена новая, поздно, следовательно, вытеснять страницы надо заранее. Это задача `paged` или своппера.

Обработка прерываний и отложенные действия

Если обработчик прерывания вызвал планировщик, то обработчик остается в системе в это время. Если планировщик

долго работает, то всё это время висит код с высочайшим уровнем приоритета. Только если это действие не выполняется как отложенное. Действия обработчика и свопера инициируются как отложенное действие. В Windows это очередь `DPC dispatch`, `deferred procedure call`. Если обработчик прерывания системного таймера будет обрабатываться длительное время, то это может сказаться на отзывчивости системы.

Пересчет динамических приоритетов

Пересчет динамических приоритетов, их имеют только процессы уровня пользователя. Процессу при создании назначается приоритет, и он является базовым для приоритета потоков. В современных системах единицей диспетчеризации является поток, так как потоки затрачивают ресурсы, но владеет ими процесс.

Факторы изменения приоритета

Приоритет пользовательских процессов пересчитывается в зависимости от времени простоя в очереди готовых процессов, от полученного процессорного времени и от того, на чем был блокирован процесс, для этого есть таблицы. Два типа потоков реального времени это аудио и видео. Пересчет приоритетов берется из Вахалии по формуле.

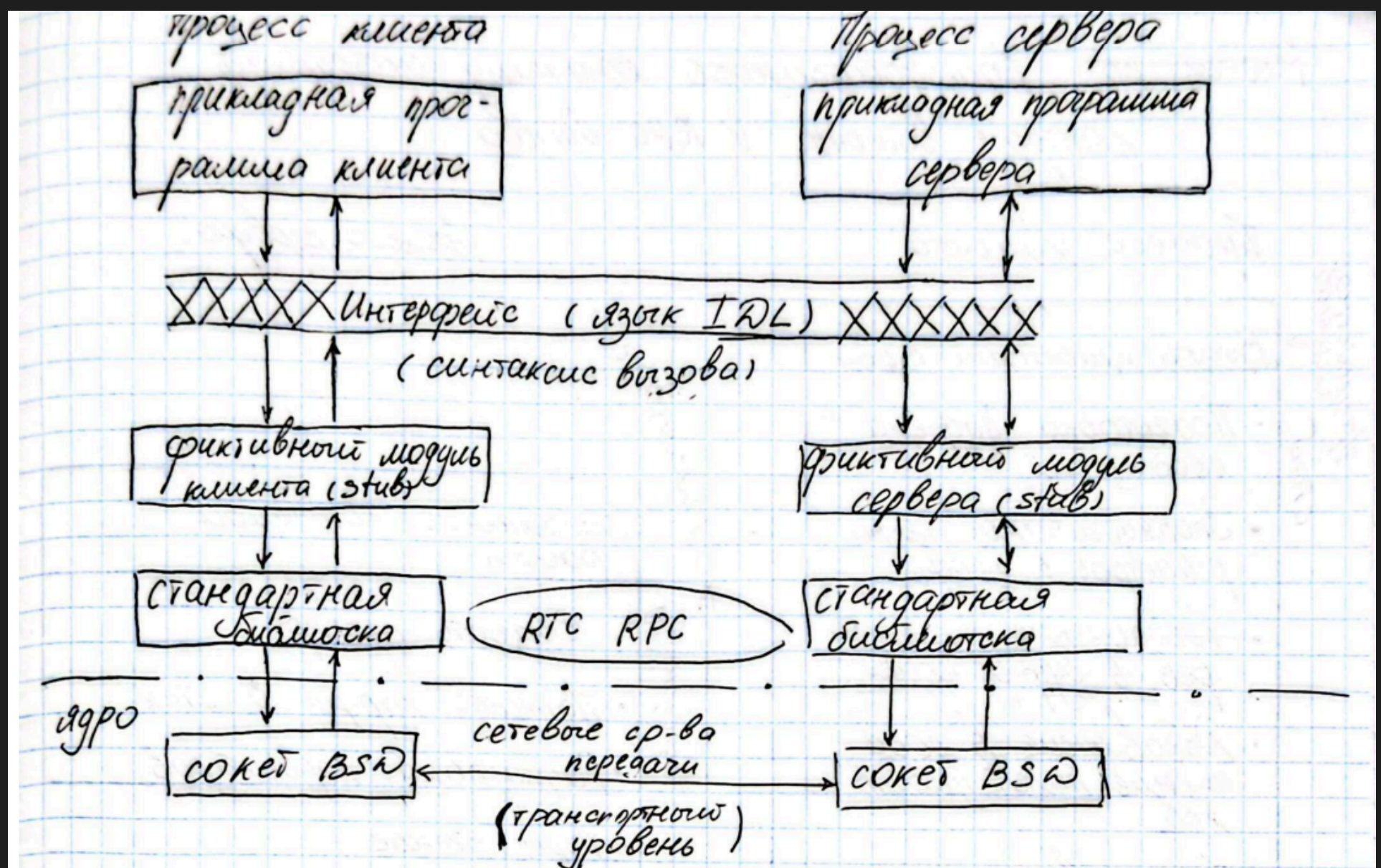
Работа с таймером в Windows

Две функции для работы с таймером Windows. Первая это регистрация хендлера или повеситься на событие.

Выводы и сравнение систем

Три вывода. В UNIX и Linux динамические приоритеты имеют только пользовательские процессы. В обеих системах учитывается время простоя потоков. Отчет с учетом потоков. Пересчета приоритетов у Стивена Раго нет. У потоков UNIX приоритет относительный, как и в Windows. В UNIX учитывается и время простоя, и время работы процессора, а в Windows только время простоя.

- **Принцип:** Удаленный вызов процедур (RPC) подобен локальному вызову процедур.
При объявлении процедуры указываются **формальные** параметры.
При вызове передаются **фактические** параметры.
- **Модель:** Клиент-Сервер. У одного сервера может быть много клиентов.
- **Гетерогенные системы:** RPC позволяет взаимодействовать разным системам (разные ОС, архитектуры).



XDR (External Data Representation)

Обеспечивает унифицированное представление данных для передачи между гетерогенными системами (решает проблемы разного порядка байт, кодировок и т.д.).

Последовательность выполнения RPC-вызова (Алгоритм)

Последовательность инструкций при выполнении удаленного вызова:

На стороне Клиента:

Вызов клиентского стаба: Приложение вызывает заглушку как обычную функцию.

Подготовка буфера сообщения: Выделение памяти под пакет.

Упорядочивание параметров (Marshalling): Упаковка параметров в буфер.

Добавление заголовков: В сообщение добавляются служебные поля.

Переход в режим ядра: Выполняется системный вызов.

Копирование в ядро: Сообщение копируется из пространства пользователя в пространство ядра.

Примечание: Ядро не может просто взять данные у пользователя; существуют спец. команды для получения информации из пространства пользователя (copy_from_user).

Определение адреса назначения: Указывается IP и порт сервера.

Установка сетевого интерфейса: Выбор маршрута/интерфейса.

Включение таймера: Клиентское приложение **блокируется** в ожидании ответа (включается таймер ожидания).

(Пакет передается по сети)

На стороне Сервера:

Аппаратное прерывание: Пакет приходит на сетевую карту.

Проверка правильности пакета: Контрольные суммы и т.д.

Определение стаба: Система определяет, какой процесс (стаб) ждет этот пакет.

Копирование данных: Сообщение копируется из ядра пользователю.

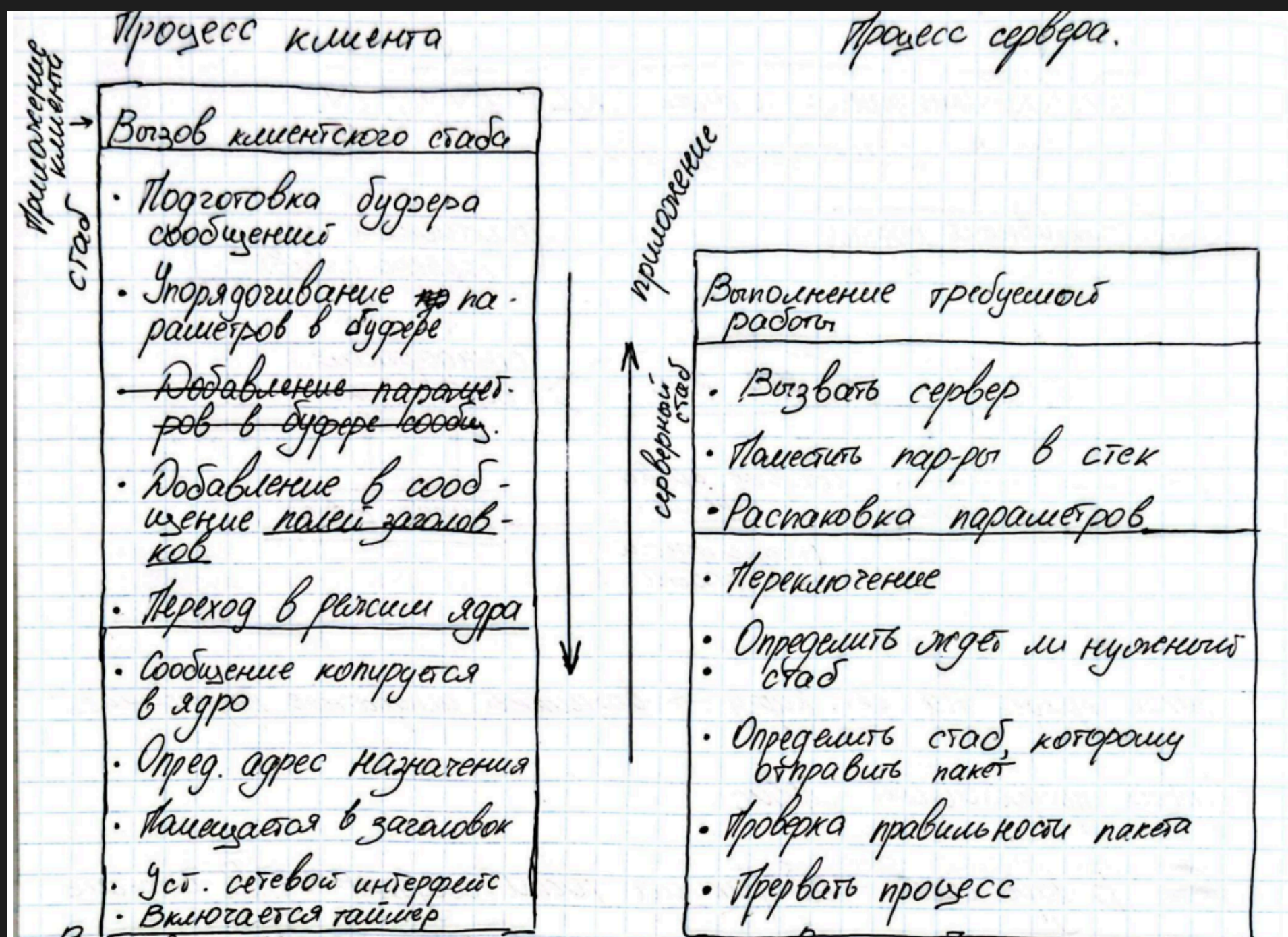
Переключение контекста: Управление передается серверному процессу.

Распаковка параметров (Unmarshalling): Извлечение аргументов из буфера.

Помещение параметров в стек: Подготовка к вызову функции.

Вызов сервера: Выполнение требуемой работы.

(После выполнения результат возвращается клиенту в обратном порядке)



Fork 5 программ лаба

C

```
#include <stdlib.h>
#include <unistd.h>
int main(void) {
    pid_t child_pids[2];
    for (int i = 0; i < 2; i++) {
        if ((child_pids[i] = fork()) == -1) {
            perror("Can not fork\n");
            return 1;
        }
        if (child_pids[i] == 0) {
            printf("Child: PID = %d, parent = %d, group = %d\n", getpid(), getppid(), getpgrp());
            sleep(2);
            printf("Child: PID = %d, parent = %d, group = %d\n", getpid(), getppid(), getpgrp());
            exit(0);
        }
    }
    printf("Parent: PID = %d, Child: PIDs = %d, %d, group = %d\n", getpid(), child_pids[0], child_pids[1], getpgrp());
    return 0;
}
```

C

```
#include <stdlib.h>
#include <sys/wait.h>
```

```

#include <unistd.h>
int main(void) {
    pid_t child_pids[2];
    const char *program_names[] = {"/matrix_mul.exe", "/read_nums.exe"};
    for (int i = 0; i < 2; i++) {
        if ((child_pids[i] = fork()) == -1) {
            perror("Can not fork\n");
            return 1;
        }
        if (child_pids[i] == 0) {
            printf("%d\n", getpid());
            if (execl(program_names[i], program_names[i], 0) == -1) {
                exit(1);
            }
        } else {
            printf("%d\n", getpid());
        }
    }
    printf("Parent: PID = %d, group = %d, Child: PIDs = %d, %d\n", getpid(), getpgrp(), child_pids[0],
child_pids[1]);
    for (int i = 0; i < 2; i++) {
        int status;
        wait(&status);
        if (WIFEXITED(status)) {
            printf("\tChild %d exited with status = %d\n", child_pids[i], WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("\tChild %d with (signal %d) killed\n", child_pids[i], WTERMSIG(status));
        } else if (WIFSTOPPED(status)) {
            printf("\tChild %d with (signal %d) stopped\n", child_pids[i], WSTOPSIG(status));
        } else {
            printf("\tUnexpected status for Child (0x%x)\n", status);
        }
    }
    return 0;
}

```

C++ ▾

```

#include <err.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>
int flag = 0;
void sig_handler(int sig_num) {
    flag = 1;
    printf("Caught signal %d\n", sig_num);
}
int main(void) {
    if (signal(SIGINT, sig_handler) == SIG_ERR) {
        err(EXIT_FAILURE, "signal");
    }
    pid_t child_pids[2];
    int pipefd[2];
    const char *messages[] = {"aaaaaaaaaaaa", "bbbbbbbbbb"};
    char buf[20];
    ssize_t num_read;
    if (pipe(pipefd) == -1)
        err(EXIT_FAILURE, "pipe");
    for (int i = 0; i < 2; i++) {
        if ((child_pids[i] = fork()) == -1) {
            perror("Can not fork\n");
            return 1;
        }
        if (child_pids[i] == 0) {
            sleep(1);
            if (flag) {

```

```

        if (close(pipefd[0]) == -1)
            err(EXIT_FAILURE, "close");
        if (write(pipefd[1], messages[i], strlen(messages[i]) + 1) == -1)
            err(EXIT_FAILURE, "write");
    }
    exit(0);
}

printf("Parent: PID = %d, group = %d, Child: PIDs = %d, %d\n", getpid(), getpgrp(), child_pids[0],
child_pids[1]);
for (int i = 0; i < 2; i++) {
    int status;
    wait(&status);
    if (WIFEXITED(status)) {
        printf("\tChild %d exited with status = %d\n", child_pids[i], WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("\tChild %d with (signal %d) killed\n", child_pids[i], WTERMSIG(status));
    } else if (WIFSTOPPED(status)) {
        printf("\tChild %d with (signal %d) stopped\n", child_pids[i], WSTOPSIG(status));
    } else {
        printf("\tUnexpected status for Child (0x%x)\n", status);
    }
}

if (close(pipefd[1]) == -1)
    err(EXIT_FAILURE, "close");
for (int i = 0; i < 2; i++) {
    num_read = read(pipefd[0], &buf, strlen(messages[i]) + 1);
    if (num_read == -1)
        err(EXIT_FAILURE, "read");
    printf("%ld, %s\n", num_read, buf);
    memcpy(buf, "\0", strlen(buf));
}

if (close(pipefd[0]) == -1)
    err(EXIT_FAILURE, "close");
return 0;
}

```

C ▾

```

#include <err.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>
int flag = 0;
void sig_handler(int sig_num) {
    flag = 1;
    printf("Caught signal %d\n", sig_num);
}

int main(void) {
    if (signal(SIGINT, sig_handler) == SIG_ERR) {
        err(EXIT_FAILURE, "signal");
    }
    pid_t child_pids[2];
    int pipefd[2];
    const char *messages[] = {"aaaaaaaaaaaa", "bbbbbbbbbb"};
    char buf[20];
    ssize_t num_read;
    if (pipe(pipefd) == -1)
        err(EXIT_FAILURE, "pipe");
    for (int i = 0; i < 2; i++) {
        if ((child_pids[i] = fork()) == -1) {
            perror("Can not fork\n");
            return 1;
        }
        if (child_pids[i] == 0) {
            sleep(1);
            if (flag) {

```

```

        if (close(pipefd[0]) == -1)
            err(EXIT_FAILURE, "close");
        if (write(pipefd[1], messages[i], strlen(messages[i]) + 1) == -1)
            err(EXIT_FAILURE, "write");
    }
    exit(0);
}

printf("Parent: PID = %d, group = %d, Child: PIDs = %d, %d\n", getpid(), getpgrp(), child_pids[0],
child_pids[1]);
for (int i = 0; i < 2; i++) {
    int status;
    wait(&status);
    if (WIFEXITED(status)) {
        printf("\tChild %d exited with status = %d\n", child_pids[i], WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("\tChild %d with (signal %d) killed\n", child_pids[i], WTERMSIG(status));
    } else if (WIFSTOPPED(status)) {
        printf("\tChild %d with (signal %d) stopped\n", child_pids[i], WSTOPSIG(status));
    } else {
        printf("\tUnexpected status for Child (0x%x)\n", status);
    }
}
if (close(pipefd[1]) == -1)
    err(EXIT_FAILURE, "close");
for (int i = 0; i < 2; i++) {
    num_read = read(pipefd[0], &buf, strlen(messages[i]) + 1);
    if (num_read == -1)
        err(EXIT_FAILURE, "read");
    printf("%ld, %s\n", num_read, buf);
    memcpy(buf, "\0", strlen(buf));
}
if (close(pipefd[0]) == -1)
    err(EXIT_FAILURE, "close");
return 0;
}

```

Теория

Системный вызов fork() создает новый процесс, который является копией процесса-предка: процесс-потомок наследует адресное пространство процесса-предка, **дескрипторы** всех открытых файлов, сигнальную маску, маску режима создания файлов и т.д.

Системный вызов fork() создает новый процесс для которого создается собственное адресное пространство, то есть таблицы страниц. Дескрипторы страниц таблиц страниц ссылаются на адресное пространство процесса-предка. При этом для страниц адресного пространства предка права доступа меняются на **only-read** и устанавливается флаг – **copy-on-write**. Если или предок, или потомок попытаются изменить страницу, возникнет исключение по правам доступа. Выполняя это исключение система обнаружит флаг copy-on-write и создаст копию страницы в адресном пространстве того процесса, который пытался ее изменить. Такая ситуация сохраняется в системе, пока потомок не вызовет `exit`.

Отношение родитель – потомок создает иерархию процессов. Если родительский процесс завершается раньше своих потомков, то в системе выполняется так называемое усыновление: процесс-потомок усыновляется процессом с идентификатором 1 (процессом «открывшим» терминал и создавшим терминальную группу) или в Ubuntu процессом-посредником `systemd –user`, который в итоге является потомком процесса с идентификатором 1. **В дескрипторе процесса содержатся указатели, обеспечивающие поддержание иерархии дерева процессов.**

Когда дочерний процесс завершается, связь его с родителем сохраняется до тех пор, пока родительский процесс в свою очередь не завершится нормально, или не вызовет `wait()` (`wait`, чтобы не возникало сирот). Следовательно, запись о дочернем процессе не исчезает из таблицы процессов немедленно. Становясь неактивным, дочерний процесс все еще остается в системе, поскольку его код завершения должен быть сохранен, на случай если родительский процесс в дальнейшем вызовет `wait()`.

Процесс-зомби – это процесс, у которого отобраны все ресурсы, кроме последнего – дескриптора процесса. Это сделано для того, чтобы процесс-предок, вызвавший системный вызов wait(), не был заблокирован навсегда.

PROCESS FLAGS top

The sum of these values is displayed in the "F" column, which is provided by the flags output specifier:

- 1 forked but didn't exec
- 4 used super-user privileges

PROCESS STATE CODES top

Here are the different values that the s, stat and state output specifiers (header "STAT" or "S") will display to describe the state of a process:

- D uninterruptible sleep (usually I/O)
- I idle kernel thread
- R running or runnable (on run queue)
- S interruptible sleep (waiting for an event to complete)
- T stopped by job control signal
- t stopped by debugger during the tracing
- W paging (not valid since Linux 2.6)
- X dead (should never be seen)
- Z defunct ("zombie") process, terminated but not reaped by its parent

For BSD formats and when the stat keyword is used, additional characters may be displayed:

- < high-priority (not nice to other users)
- N low-priority (nice to other users)
- L has pages locked into memory (for real-time and custom I/O)
- s is a session leader
- l is multi-threaded (using CLONE_THREAD, like NPTL pthreads do)
- + is in the foreground process group

Системный вызов `wait()` блокирует родительский процесс до момента завершения дочернего. При этом процесс-предок получает статус завершения процесса-потомка.

WIFEXITED(status)	Ненулевой, если дочерний процесс завершен нормально
WEXITSTATUS(stat_val)	Если WIFEXITED ненулевой, возвращает код завершения дочернего процесса
WIFSIGNALED(stat_val)	Ненулевой, если дочерний процесс завершается перехватываемым сигналом
WTERMSIG(stat_val)	Если WIFSIGNALED ненулевой, возвращает номер сигнала
WIFSTOPPED(stat_val)	Ненулевой, если дочерний процесс остановился
WSTOPSIG(stat_val)	Если WIFSTOPPED ненулевой, возвращает номер сигнала

Системный вызов `exec()` создает таблицу страниц для адресного пространства программы, переданной ему в качестве аргумента, а затем заменяет старый адрес новой таблицы страниц. (Системный вызов, который переводит процесс на адресное пространство программы, которая была передана ему в качестве аргумента).

Системный вызов `exec()` выполняет следующие действия:

1. Разбирает путь к исполняемому файлу и осуществляет доступ к нему.
2. Проверяет, имеет ли вызывающий процесс полномочия на выполнение файла.
3. Читает заголовок и проверяет, что он действительно исполняемый.

4. Если для файла установлены биты SUID или SGID, то эффективные идентификаторы UID и GID вызывающего процесса изменяет на UID и GID, соответствующие владельцу файла.
5. Копирует аргументы, передаваемые в `exec`, а также переменные среды в пространство ядра, после чего текущее пользовательское пространство готово к уничтожению.
6. Выделяет пространство своппинга для областей данных и стека.
7. Высвобождает старое адресное пространство и связанное с ним пространство свопинга. Если же процесс был создан при помощи `vfork`, производится возврат старого адресного пространства родительскому процессу.
8. Выделяет карты трансляции адресов для нового текста, данных и стека.
9. Устанавливает новое адресное пространство. Если область текста активна, то она будет совместно использоваться с этим процессом. В других случаях пространство должно инициализироваться из исполняемого файла. Процессы в системе UNIX обычно разбиты на страницы, что означает, что каждая страница считывается в память только по мере необходимости.
10. Копирует аргументы и переменные среды обратно в новый стек приложения.
11. Сбрасывает все обработчики сигналов в действия, определенные по умолчанию, так как функции обработчиков сигналов не существуют в новой программе. Сигналы, которые были проигнорированы или заблокированы перед вызовом `exec`, остаются в тех же состояниях.
12. Инициализирует аппаратный контекст. При этом большинство регистров сбрасывается в 0, а указатель команд получает значение точки входа программы. В регистр CR3 загружается адрес таблиц страниц из дескриптора процесса. В IP загружается адрес точки входа.

Системный вызов **`pipe()`** создает неименованный программный канал.

`pipe` - буфер в области данных ядра системы размером в 1 страницу.

UNIX поддерживает два типа программных каналов: именованные (`mknod <имя> p`, имеет имя), неименованные (имени файла не имеет, но имеет дескриптор).

Является единственным способом взаимодействия параллельных процессов, поддерживаемым файловой системой.

`pipe` работает в одну сторону (потокковая модель передачи данных/симплексный метод).

Считанное сообщение перестает существовать.

Системе важно, чтобы у буфера был читатель, если нет процесса, читающего из буфера, то писатели будут заблокированы.

В случае неименованного программного канала сообщения могут передавать только процессы-родственники, так как **`fork()`** создает процесс, который наследует дескрипторы файлов, не имя, так как оно неоднозначно определяет файл.

Буферизация на 3-х уровнях:

1. Буфер в системной области памяти, размер его установлен в системе: 4096 байт — 1 страница.
2. Программные каналы буферизуются во вторичной памяти. Если очень много `pipe`, то самые старые переписываются на диск.
3. Буферизуются во времени. Если процесс пытается записать больше, чем 4096 байт, то он блокируется, пока не будут прочитаны данные.

Сигналы - базовое свойство информирования процессов о событиях, которые могут происходить как вне самих процессов, так и в них самих. (Ход выполнения программы можно изменить с помощью внешнего асинхронного события).

Если событие произошло вне самого процесса, то это внешнее асинхронное событие.

Событие, произошедшее в процессе выполнения кода, является синхронным событием.

Реакция процесса на событие:

1. Прогнорировать полученный сигнал.
2. Реагировать на сигнал по-умолчанию (способом, определенным в системе).
3. Процесс может определить свою реакцию на полученный сигнал, следовательно требуется регистрация в системе своего обработчика сигнала.


```

        {ACTIVE_WRITER, 0, 0},
        {ACTIVE_WRITER, V, 0},
        {TURN_WRITE, P, 0} };

struct sembuf stop_write[] = { {ACTIVE_WRITER, P, 0} };
int count = 0;

int flag = 1;
void sig_handler(int sig_num)
{
    flag = 0;
}

int reader(int semfd, char *addr)
{
    while (flag)
    {
        if (semop(semfd, start_read, 5) == -1)
        {
            printf("Reader %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        printf("Reader %d - %c\n", getpid(), *addr);

        if (semop(semfd, stop_read, 1) == -1)
        {
            printf("Reader %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

int writer(int semfd, char *addr)
{
    while (flag)
    {
        if (semop(semfd, start_write, 5) == -1)
        {
            printf("Writer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        if (*addr == 'z')
        {
            *addr = 'a';
            count++;
        }
        else
        {
            (*addr)++;
        }
        if (count == 2)
        {
            if (kill(0, SIGALRM) == -1)
            {
                perror("kill");
                exit(1);
            }
            exit(0);
        }

        printf("Writer %d - %c\n", getpid(), *addr);

        if (semop(semfd, stop_write, 1) == -1)
        {
            printf("Writer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));

```

```

        exit(1);
    }
}

exit(0);
}

int main(void)
{
    int shmfd, semfd;
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    char *addr;
    char **pr_ptr;
    char **cn_ptr;
    char *letter;
    pid_t chpid[5];

    semfd = semget(100, 4, IPC_CREAT | perms);
    if (semfd == -1)
    {
        perror("semget");
        exit(1);
    }

    if (semctl(semfd, ACTIVE_READER, SETVAL, 0) == -1)
    {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, ACTIVE_WRITER, SETVAL, 0) == -1)
    {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, TURN_WRITE, SETVAL, 0) == -1)
    {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, TURN_READ, SETVAL, 0) == -1)
    {
        perror("semctl");
        exit(1);
    }

    shmfd = shmget(200, sizeof(char), IPC_CREAT | perms);
    if (shmfd == -1)
    {
        perror("shmget");
        exit(1);
    }

    addr = shmat(shmfd, NULL, 0);
    if (addr == (char *) -1)
    {
        perror("shmat");
        exit(1);
    }

    if (signal(SIGALRM, sig_handler) == SIG_ERR)
    {
        perror("signal");
        exit(1);
    }
}

```

```

*addr = 'a';

for (int i = 0; i < 3; i++)
{
    chpid[i] = fork();
    if (chpid[i] == -1)
    {
        perror("fork");
        exit(1);
    }

    if (chpid[i] == 0)
    {
        writer(semfd, addr);
    }
}

for (int i = 3; i < 5; i++)
{
    chpid[i] = fork();
    if (chpid[i] == -1)
    {
        perror("fork consumer");
        exit(1);
    }

    if (chpid[i] == 0)
    {
        reader(semfd, addr);
    }
}

for (int i = 0; i < 5; i++)
{
    int status;
    pid_t child = wait(&status);

    if (WIFEXITED(status))
        printf("Child: PID = %d exited with status = %d\n", child, WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("Child: PID = %d killed by signal = %d\n", child, WTERMSIG(status));
    else if (WIFSTOPPED(status))
        printf("Child: PID = %d stopped by signal = %d\n", child, WSTOPSIG(status));
    else
        printf("Child has finished: PID = %d\n", child);
}

if (shmdt(addr) == -1)
{
    perror("shmdt");
    exit(1);
}

if (semctl(semfd, IPC_RMID, 0) == -1)
{
    perror("semctl");
    exit(1);
}

if (shmctl(shmfd, IPC_RMID, NULL) == -1)
{
    perror("shmctl");
    exit(1);
}

return 0;
}

```

C ▾

```
#include <stdio.h>
#include <time.h>
#include <windows.h>

HANDLE can_read;
HANDLE can_write;
LONG wrt = 0;
HANDLE mutex;
LONG count_writers = 0;
LONG count_readers = 0;
LONG active_reader = 0;
int flag = 1;
char letter = 'a';
int count = 0;

void start_read()
{
    InterlockedIncrement(&count_readers);
    if (wrt || count_writers)
    {
        WaitForSingleObject(can_read, INFINITE);
    }
    WaitForSingleObject(mutex, INFINITE);
    InterlockedDecrement(&count_readers);
    InterlockedIncrement(&active_reader);
    if(!SetEvent(can_read))
    {
        perror("SetEvent");
        ExitThread(1);
    }
    if (!ReleaseMutex(mutex))
    {
        perror("ReleaseMutex");
        ExitThread(1);
    }
}

void stop_read()
{
    InterlockedDecrement(&active_reader);
    if (active_reader == 0)
    {
        if (!SetEvent(can_write))
        {
            perror("SetEvent");
            ExitThread(1);
        }
    }
}

void start_write()
{
    InterlockedIncrement(&count_writers);
    if (active_reader > 0 || wrt)
    {
        WaitForSingleObject(can_write, INFINITE);
    }
    InterlockedDecrement(&count_writers);
    wrt = 1;
    if (!ResetEvent(can_write))
    {
        perror("ResetEvent");
        ExitThread(1);
    }
}
```

```

void stop_write()
{
    wrt = 0;
    if (count_readers > 0)
    {
        if (!SetEvent(can_read))
        {
            perror("SetEvent");
            ExitThread(1);
        }
    }
}
else
{
    if (!SetEvent(can_write))
    {
        perror("SetEvent");
        ExitThread(1);
    }
}
}

DWORD reader(LPVOID param)
{
    while (flag)
    {
        start_read();
        printf("R %d - %c\n", GetCurrentThreadId(), letter);
        stop_read();
    }
    ExitThread(0);
}

DWORD writer(LPVOID param)
{
    while (flag)
    {
        if (letter == 'z')
        {
            count++;
            letter = 'a' - 1;
        }
        if (count == 2)
        {
            GenerateConsoleCtrlEvent(CTRL_C_EVENT, 0);
        }
        printf("W %d - %c\n", GetCurrentThreadId(), ++letter);
        stop_write();
    }
    ExitThread(0);
}

BOOL sig_handler(DWORD sig_num)
{
    flag = 0;
}

int main()
{
    int readers_amt = 5, writers_amt = 3;
    HANDLE readers[readers_amt], writers[writers_amt];
    if ((mutex = CreateMutex(NULL, FALSE, NULL)) == NULL)
    {
        perror("CreateMutex");
        exit(1);
    }
    if ((can_read = CreateEvent(NULL, FALSE, FALSE, NULL)) == NULL)
    {
        perror("CreateEvent");
    }
}

```

```

        exit(1);
    }
    if ((can_write = CreateEvent(NULL, TRUE, FALSE, NULL)) == NULL)
    {
        perror("CreateEvent");
        exit(1);
    }
    if (!SetConsoleCtrlHandler(sig_handler, TRUE))
    {
        perror("SetConsoleCtrlHandler");
        exit(1);
    }

    DWORD writers_id[writers_amt];
    for (int i = 0; i < writers_amt; i++)
    {
        writers_id[i] = i;
        if ((writers[i] = CreateThread(NULL, 0, writer, NULL, 0, &writers_id[i])) == NULL)
        {
            perror("CreateThread");
            exit(1);
        }
    }

    DWORD readers_id[readers_amt];
    for (int i = 0; i < readers_amt; i++)
    {
        readers_id[i] = i;
        if ((readers[i] = CreateThread(NULL, 0, reader, NULL, 0, &readers_id[i])) == NULL)
        {
            perror("CreateThread");
            exit(1);
        }
    }

    WaitForMultipleObjects(readers_amt, readers, TRUE, INFINITE);
    WaitForMultipleObjects(writers_amt, writers, TRUE, INFINITE);
    DWORD rc;
    for (int i = 0; i < readers_amt; i++)
    {
        GetExitCodeThread(readers[i], &rc);
        printf("Reader %d exited with code %d\n", readers_id[i], rc);
        CloseHandle(readers[i]);
    }
    for (int i = 0; i < writers_amt; i++)
    {
        GetExitCodeThread(writers[i], &rc);
        printf("Writer %d exited with code %d\n", writers_id[i], rc);
        CloseHandle(writers[i]);
    }
    if (!CloseHandle(can_read))
    {
        perror("CloseHandle");
        ExitThread(1);
    }
    if (!CloseHandle(can_write))
    {
        perror("CloseHandle");
        ExitThread(1);
    }
    if (!CloseHandle(mutex))
    {
        perror("CloseHandle");
        ExitThread(1);
    }
}

```

Мониторы Хоара — средство обеспечения монопольного доступа более высокого уровня. Мониторы были предложены для того, чтобы структурировать свойства взаимoisключения.

Монитор — высокоуровневое средство взаимоисключения, может быть языковой конструкцией, может быть реализован средствами ОС. Представляет из себя набор защищенных данных и процедур, причем к данным монитора могут обращаться только процедуры монитора.

На переменной типа условие (conditional) определены два примитива:

C ▾

```
wait()
signal()
```

Для того, чтобы обратиться к данным монитора, вызываются функции монитора.

Процесс, вызвавший функцию монитора, считается находящимся в мониторе.

Сам монитор — ресурс, к нему может быть организована очередь.

Pascal ▾

```
resource: monitor;
var
    busy: logical;
    x: conditional;
procedure acquire; //захватить монитор (1)
begin
    if busy then wait(x);
    busy := true;
end;

procedure release; // освободить монитор (2)
begin
    busy := false;
    signal(x);
end;

begin // инициализация
    busy := false;
end;
```

(1) Процессу нужен ресурс, если ресурс занят, то он блокируется системным вызовом wait() на переменной типа условие. Свободен — меняет значение переменной busy = true, ресурс занят.

(2) Процесс освобождает ресурс; busy = false, вызывает системный вызов signal(x) — просигнализирует, что ресурс освободился. Разблокировка процесса, находящегося в очереди к монитору.

Читатели-писатели. Читатели только читают данные, могут читать параллельно. Писатели могут изменять данные, должны работать в режиме монопольного доступа.

Читатель может начать читать, если нет активного писателя и очередь писателей пуста. В обратном случае читатель переходит в состояние ожидания. Читать изменяемые данные нельзя. Цепная реакция читателей.

Конец чтения, декремент числа активных читателей, если нет активных читателей, то посылается сигнал можно писать.

Писатель может начать писать, если нет активных читателей и нет активного писателя. В обратном случае, блокирован на переменной типа условие "можно писать". Писатель захватывает логическую переменную, присваивая ей значение true.

Заканчивает писать, wrt = false. Если есть очередь ждущих читателей, то посылается сигнал "можно читать", иначе сигнал "можно писать".

Монитор написан таким образом, чтобы исключить бесконечное откладывание читателей и писателей.

В С нет системного вызова turn, следовательно реализуется на трех счетчиках: ждущих читателей, активных читателей, ждущих писателей. Одна логическая переменная писатель (бинарный семафор).

Когда читатель вызывает start_read, то увеличивается число ждущих читателей, если они успешно проходят условный оператор, то читатель из ждущего переходит в активного, когда прочитал, то число активных уменьшается.

Писатель, когда вызывает start_write, то увеличивает число ждущих писателей, при захвате wrt число ждущих писателей уменьшается.

Pascal ▾

```
resource: monitor;
var
    nr: integer; //читатели
    wrt: logical; // писатель
    c_read, c_write: conditional;
procedure start_read;
begin
    if wrt or turn(c_write)
        then wait(c_read);
    nr := nr + 1;
    signal(c_read);
end;
procedure stop_read;
begin
    nr := nr - 1;
    if nr = 0 then signal(c_write);
end;
procedure start_write;
begin
    if nr > 0 or wrt then wait(c_write);
    wrt := true;
end;
procedure stop_write;
begin
    wrt := false;
    if turn(c_read) then signal(c_read);
    else signal(c_write);
end;

begin // инициализация
    nr := 0;
    wrt := false;
end;
```

Производство-потребление (кольцевой буфер).

Буфер представлен в виде массива (сделать из массива кольцо).

На кольцевом буфере определены две переменные типа условие:

1. buffer_full;
2. buffer_empty;

Pascal ▾

```
resource: monitor;
var
    bcircle: array[0..n-1] of <type>
    pos: 0..n; // текущая позиция
    j: 0..n-1; // заполняемая позиция
    x: 0..n-1; // освобождаемая позиция
    buffer_full, buffer_empty: conditional;

procedure producer(data: <type>);
begin
    if pos = n then wait(buffer_empty);
    bcircle[j] := data; // положить данные
    pos := pos + 1; // инкремент текущей позиции
    j := (j + 1) mod n; // заикливание массива
    signal(buffer_full); // (1)
end;

procedure consumer(var data: <type>);
begin
    if pos = 0 then wait(buffer_full);
    data := bcircle[k]; // выбор данных
    pos := pos - 1; // декремент текущей позиции
```

```

        k := (k + 1) mod n;
        signal(buffer_empty); //(2)
    end;

begin // инициализация
    pos := 0;
    j := 0;
    k := 0;
end;

```

(1) Сигнал потребителю, что буффер заполнен. Процесс, желающий положить, что-то в хранилище вызывает процедуру produce, текущая позиция равна n, следовательно буффер заполнен, процесс перейдет в состояние блокировки.

(2) Сигнал процессу, который может быть заблокирован на полностью заполненном хранилище.

Производство-потребление

C ▾

```

#include <stdlib.h>
#include <string.h>
#include <sys/errno.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#define SB 0
#define SE 1
#define SF 2
#define P -1
#define V 1
struct sembuf start_produce[2] = {{SE, P, 0}, {SB, P, 0}};
struct sembuf stop_produce[2] = {{SB, V, 0}, {SF, V, 0}};
struct sembuf start_consume[2] = {{SF, P, 0}, {SB, P, 0}};
struct sembuf stop_consume[2] = {{SB, V, 0}, {SE, V, 0}};
int count = 0;

int flag = 1;
void sig_handler(int sig_num) {
    flag = 0;
}

void producer(int semfd, char **pr_ptr, char *letter) {
    while (flag) {
        if (semop(semfd, start_produce, 2) == -1) {
            printf("Producer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        if (*letter == 'z') {
            *letter = 'a';
            count++;
        } else {
            (*letter)++;
        }
        (*pr_ptr)++;
        if (count == 2) {
            if (kill(0, SIGALRM) == -1) {
                perror("kill");
                exit(1);
            }
            exit(0);
        }

        **pr_ptr = *letter;
    }
}

```

```

        printf("Producer %d - %c\n", getpid(), **pr_ptr);

        if (semop(semfd, stop_produce, 2) == -1) {
            printf("Producer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

void consumer(int semfd, char **cn_ptr) {
    while (flag) {
        if (semop(semfd, start_consume, 2) == -1) {
            printf("Consumer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        (*cn_ptr)++;
        printf("Consumer %d - %c\n", getpid(), **cn_ptr);

        if (semop(semfd, stop_consume, 2) == -1) {
            printf("Consumer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

int main() {
    int shmfd, semfd;
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    char *addr;
    char **pr_ptr;
    char **cn_ptr;
    char *letter;
    pid_t chpid[7];

    semfd = semget(100, 3, IPC_CREAT | perms);
    if (semfd == -1) {
        perror("semget");
        exit(1);
    }

    if (semctl(semfd, SB, SETVAL, 1) == -1) {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, SE, SETVAL, 256) == -1) {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, SF, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }

    shmfd = shmget(101, 256, IPC_CREAT | perms);
    if (shmfd == -1) {
        perror("shmget");
        exit(1);
    }

    addr = shmat(shmfd, NULL, 0);
    if (addr == (char *)-1) {

```

```

        perror("shmat");
        exit(1);
    }

    pr_ptr = (char **)addr;
    cn_ptr = pr_ptr + 1;
    letter = (char *) (cn_ptr + 1);

    *cn_ptr = letter + 1;
    *pr_ptr = *cn_ptr;
    *letter = 'a' - 1;

    if (signal(SIGALRM, sig_handler) == SIG_ERR) {
        perror("signal");
        exit(1);
    }

    for (int i = 0; i < 3; i++) {
        chpid[i] = fork();
        if (chpid[i] == -1) {
            perror("fork");
            exit(1);
        }

        if (chpid[i] == 0) {
            producer(semfd, pr_ptr, letter);
        }
    }

    for (int i = 3; i < 7; i++) {
        chpid[i] = fork();
        if (chpid[i] == -1) {
            perror("fork consumer");
            exit(1);
        }

        if (chpid[i] == 0) {
            consumer(semfd, cn_ptr);
        }
    }

    for (int i = 0; i < 7; i++) {
        int status;
        pid_t child = wait(&status);

        if (WIFEXITED(status))
            printf("Child: PID = %d exited with status = %d\n", child, WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("Child: PID = %d killed by signal = %d\n", child, WTERMSIG(status));
        else if (WIFSTOPPED(status))
            printf("Child: PID = %d stopped by signal = %d\n", child, WSTOPSIG(status));
        else
            printf("Child has finished: PID = %d\n", child);
    }

    if (shmdt(addr) == -1) {
        perror("shmdt");
        exit(1);
    }

    if (semctl(semfd, 0, IPC_RMID, NULL) == -1) {
        perror("semctl");
        exit(1);
    }

    if (shmctl(shmfd, IPC_RMID, NULL) == -1) {
        perror("shmctl");
        exit(1);
    }

```

```
}

return 0;
}
```

Теория

Формулировка: Есть два типа процессов: первый тип — процессы-производители, которые только производят единицу данных и помещают в ячейки буфера, второй тип — процессы-потребители, которые могут только выбирать данные из буфера.

Производители-потребители — асинхронные процессы, каждый выполняется с собственной скоростью. Если буфер (хранилище) заполнено, то производитель будет блокирован. Если производитель запаздывает, то потребитель ждет производителя. Если ячейка освобождена, то второй раз взять значение из этой же ячейки невозможно.

Семафор — защищенная неотрицательная переменная, на которой определены две операции: P — декремент, захват семафора, V — инкремент, освобождение семафора.

Виды:

- 1. Бинарный семафор (0/1).
- 2. Считающий > 0, int.

Алгоритм Деикстры решения задачи производства-потребления:
Se, Sf — считающие семафоры, Sb — бинарный.

```
C ~

Producer:
    P(Se)
    P(Sb)
    N = N + 1;
    V(Sb)
    V(Sf)

Consumer:
    P(Sf)
    P(Sb)
    N = N - 1;
    V(Sb)
    V(Se)
```

Sb обеспечивает монопольный доступ к буферу.

Основная особенность семафоров UNIX в том, что одной неделимой операцией можно изменить все или часть семафоров набора.

Система должна предоставить соответствующие системные вызовы, система переходит в режим ядра, что связано с переключением аппаратного контекста.

Структуры для работы с семафорами — структуры ядра. В ядре поддержка с помощью таблицы семафоров, содержащей дескриптор набора (semid_ds).

О каждом наборе семафоров известно:

- 1. Имя — целое число. По этому имени другие процессы могут получить дескриптор набора.
- 2. UID создателя набора семафоров и GID. Процесс, эффективный UID которого совпадает с UID создателя может удалять набор и исправлять его управляющие параметры.
- 3. Права доступа.
- 4. Количество семафоров в наборе.
- 5. Время изменения одного или нескольких значений семафоров набора.
- 6. Время последнего изменения управляющих параметров набора.
- 7. Указатель на массив семафоров.

На семафорах определены следующие системные вызовы:

C ▾

```
#include <sys/sem.h>

int semget(key_t key, int nsems, int semflg); //создание набора семафоров, где key -- идентификатор набора,
который указывает процесс при создании набора семафоров; nsems -- число семафоров в наборе; flag -- флаги.
int semctl(int semid, int semnum, int op, ...);
int semop(int semid, struct sembuf *sops, size_t nsops);
struct sembuf
{
    unsigned short sem_num; /* semaphore number */
    short          sem_op;  /* semaphore operation */
    short          sem_flg; /* operation flags */
};
```

Флаги:

1. IPC_NOWAIT указывает, что процесс не желает переходить в состояние ожидания освобождения семафора. Это сделано для того, чтобы все процессы, заблокированные к семафору, при получении сигнала kill процессом, удерживающим семафор, который нельзя перехватить, в результате чего процесс не сможет освободить семафор, не были заблокированы навечно.
2. SEM_UNDO — этот флаг указывает системе на необходимость отслеживать изменения значения семафора операцией sem_op и при завершении вызывающего процесса ликвидировать сделанные им изменения. Процессы не будут заблокированы семафором навечно.

Основное отличие семафора от мьютекса:

У семафора нет хозяина, его может освободить любой другой процесс, который знает его идентификатор, в отличие от мьютекса, у которого есть хозяин, его может освободить процесс, который его захватил.

Сегменты разделяемой памяти:

Определение: Буфер в области данных ядра системы. Так как у процессов защищенное виртуальное адресное пространство, то их взаимодействие возможно только через третье адресное пространство.

На `pipe` определен массив файловых дескрипторов, который контролирует доступ (на сегментах никакого контроля нет, это буфер, вместе с разделяемым сегментом используют семафоры, которые регулируют доступ процессов к разделяемому сегменту).

Информация, записываемая в разделяемый сегмент после чтения не исчезает, следовательно разные процессы могут прочитать одну и ту же информацию.

Сегменты разделяемой памяти поддерживаются при помощи таблицы разделяемых сегментов в ядре. Их особенность заключается в том, что разделяемый сегмент подключается к адресному пространству процесса. За счет этого исключается копирование.

Системные вызовы:

C ▾

```
#include <sys/shm.h>
```

```
int shmget(key_t key, size_t size, int shmflg);
```

`shmget()` returns the identifier of the System V shared memory segment associated with the value of the argument `key`.

```
int shmctl(int shmid, int op, struct shmid_ds *buf);
```

`shmctl()` performs the control operation specified by `op` on the System V shared memory segment whose identifier is given in `shmid`.

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

The `shmat()` function attaches the shared memory segment associated with the shared memory identifier specified by `shmid` to the address space of the calling process.

```
int shmdt(const void *shmaddr);
```

The `shmdt()` function detaches the shared memory segment located at the address specified by `shmaddr` from the address space of the calling process.

Демон

C ▾

```
#include <errno.h>
#include <fcntl.h>
#include <pthread.h>
#include <sched.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <sys/stat.h>
#include <sys/syslog.h>
#include <time.h>
#include <unistd.h>
sigset_t mask;
void daemonize(const char* cmd)
{
    int i, fd0, fd1, fd2;
    pid_t pid;
    struct rlimit rl;
    struct sigaction sa;
    umask(0);
    if (getrlimit(RLIMIT_NOFILE, &rl) == -1)
    {
        perror("getrlimit");
        exit(1);
    }
    if ((pid = fork()) == -1)
    {
        perror("fork");
        exit(1);
    }
    else if (pid != 0)
    {
        exit(0);
    }
    if (setsid() == -1)
    {
        perror("setsid");
        exit(1);
    }
}
```



```

    write(fd, buf, strlen(buf) + 1);
    return 0;
}

void reread(void)
{
    int fd;
    if ((fd = open("/etc/daemon.conf", O_RDONLY)) == -1)
    {
        syslog(LOG_ERR, "open");
        exit(1);
    }
    if (read(fd, NULL, 0) == -1)
    {
        syslog(LOG_ERR, "read");
        exit(1);
    }
    if (close(fd) == -1)
    {
        syslog(LOG_ERR, "close");
        exit(1);
    }
}

void* thr_fn(void* arg)
{
    int err, signo;
    for (;;)
    {
        err = sigwait(&mask, &signo);
        if (err > 0)
        {
            syslog(LOG_ERR, "ошибка вызова функции sigwait");
            exit(1);
        }
        switch (signo)
        {
            case SIGHUP:
                syslog(LOG_INFO, "Чтение конфигурационного файла");
                reread();
                break;
            case SIGTERM:
                syslog(LOG_INFO, "получен сигнал SIGTERM; выход");
                remove("/var/run/daemon.pid");
                pthread_exit(NULL);
            default:
                syslog(LOG_INFO, "получен сигнал %d\n", signo);
        }
    }
}

int main(int argc, char* argv[])
{
    int err;
    pthread_t tid;
    char* cmd;
    struct sigaction sa;
    if ((cmd = strchr(argv[0], '/')) == NULL)
    {
        cmd = argv[0];
    }
    else
    {
        cmd++;
    }
    daemonize(cmd);
    if (already_running())
    {
        syslog(LOG_ERR, "already_running");
        exit(1);
    }
}

```

```

sa.sa_handler = SIG_DFL;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
if (sigaction(SIGHUP, &sa, NULL) == -1)
{
    syslog(LOG_SYSLOG, "sigaction");
}
if (sigfillset(&mask) == -1)
{
    perror("sigfillset");
    exit(1);
}
if ((err = pthread_sigmask(SIG_BLOCK, &mask, NULL)) == -1)
{
    perror("pthread_sigmask");
    exit(1);
}
err = pthread_create(&tid, NULL, thr_fn, 0);
if (err != 0)
{
    perror("pthread_create");
    exit(1);
}
time_t raw_time;
struct tm* timeinfo;
for (;;)
{
    sleep(5);
    time(&raw_time);
    timeinfo = localtime(&raw_time);
    syslog(LOG_INFO, "%s", asctime(timeinfo));
}
}

```

Теория

Демоны — это долгоживущие процессы. Так как они не имеют управляющего терминала, говорят, что они работают в фоновом режиме.

Правила программирования демонов:

1. Вызвать функцию `umask`, чтобы сбросить маску режима создания файлов в значение 0. Маска, наследуемая от запускающего процесса, может маскировать некоторые биты прав доступа. Если предполагается, что процесс-демон будет создавать файлы, может потребоваться установка определенных битов прав доступа. Например, если демон создает файлы с правом на чтение и на запись для группы, маска режима создания файла, которая включает любой из этих битов, будет препятствовать этому. С другой стороны, если демон вызывает библиотечные функции, создающие файлы, имеет смысл установить более ограничивающую маску (например, 007), так как библиотечные функции могут не принимать аргумент с битами прав доступа.
2. Вызвать функцию `fork` и завершить родительский процесс. Для чего это делается? Во-первых, если демон запущен как обычная команда оболочки, завершив родительский процесс, мы заставим командную оболочку думать, что команда выполнена. Во-вторых, дочерний процесс наследует идентификатор группы процессов от родителя, но получает свой идентификатор процесса; тем самым гарантируется, что дочерний процесс не будет являться лидером группы, а это необходимое условие для вызова функции `setuid`, который будет произведен далее.
3. Создать новый сеанс, обратившись к функции `setsid`. При этом процесс становится (а) лидером нового сеанса, (б) лидером новой группы процессов и (в) лишается управляющего терминала.
4. Сделать корневой каталог текущим рабочим каталогом. Текущий рабочий каталог, унаследованный от родительского процесса, может находиться на смонтированной файловой системе. Поскольку демон, как правило, существует все время, пока система не перезагрузится, в подобной ситуации, когда рабочий каталог демона находится в смонтированной файловой системе, ее невозможно будет отмонтировать. Как вариант, некоторые демоны могут устанавливать собственный текущий рабочий каталог, в котором они производят все необходимые действия. Например, демоны печати в качестве текущего рабочего каталога часто выбирают буферный каталог, куда помещаются задания для печати.
5. Закрыть все ненужные файловые дескрипторы. Это предотвращает удержание в открытом состоянии некоторых дескрипторов, унаследованных от родительского процесса (командной оболочки или другого процесса). С помощью

функции `getrlimit` можно определить максимально возможный номер дескриптора и закрыть все дескрипторы вплоть до этого номера.

6. Некоторые демоны открывают файловые дескрипторы с номерами 0, 1 и 2 на устройстве `/dev/null`, — таким образом, любые библиотечные функции, которые пытаются читать со стандартного устройства ввода или писать в стандартное устройство вывода или сообщений об ошибках, не будут оказывать никакого влияния. Поскольку демон не связан ни с одним терминальным устройством, он не сможет взаимодействовать с пользователем в интерактивном режиме. Даже если демон запущен в интерактивном сеансе, он все равно переходит в фоновый режим, и начальный сеанс может завершиться без воздействия на процесс-демон. С этого же терминала в систему могут входить другие пользователи, и демон не должен выводить какую-либо информацию на терминал, да и пользователи не ждут, что их ввод с терминала будет прочитан демоном.

Соглашения для демонов:

1. Если демон использует файл блокировки, этот файл помещается в каталог `/var/run`. Однако, чтобы создать файл в этом каталоге, демон должен обладать привилегиями суперпользователя. Имя файла обычно имеет вид `name.pid`, где `name` — имя демона или службы. Например, демон `crond` создает файл блокировки с именем `/var/run/crond.pid`.
2. Если демон поддерживает определение дополнительных настроек, они обычно хранятся в каталоге `/etc`. Имя конфигурационного файла, как правило, имеет вид `name.conf`, где `name` — имя демона или службы. Например, конфигурационный файл демона `syslogd` называется `/etc/syslog.conf`.
3. Демоны могут запускаться из командной строки, но чаще запуск демонов производится из сценариев инициализации системы (`/etc/rc*` или `/etc/init.d/*`). Если после завершения демон должен автоматически перезапускаться, мы можем указать на это процессу `init`, добавив запись `respawn` в файл `/etc/inittab` (предполагается, что система использует команду `init` в стиле System V).
4. Если демон имеет конфигурационный файл, настройки из него читаются демоном во время запуска, и затем он обычно не обращается к этому файлу. Если в конфигурационный файл были внесены изменения, демон пришлось бы останавливать и перезапускать снова, чтобы новые настройки вступили в силу. Во избежание этого некоторые демоны устанавливают обработчики сигнала `SIGHUP`, в которых производится чтение конфигурационного файла и перенастройка демона. Поскольку демоны не имеют управляющего терминала и являются либо лидерами сеансов без управляющего терминала, либо членами осиротевших групп процессов, у них нет причин ожидать сигнала `SIGHUP`. Поэтому он может использоваться для других целей.

Структура сигналов:

C ▾

```
#include <signal.h>
```

```
int sigaction(int signum, const struct sigaction *Nullable restrict act, struct sigaction *Nullable restrict oldact);
```

```
struct sigaction {
    void      (*sa_handler)(int);
    void      (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t   sa_mask;
    int        sa_flags;
    void      (*sa_restorer)(void);
};
```

On some architectures a `union` is involved: do not assign to both `sa_handler` and `sa_sigaction`.

```
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
```

`SIG_BLOCK` The set of blocked signals is the `union` of the current set and the set argument.

Вопросы:

Что такое `SIG_BLOCK`? — Объединение текущей и передаваемой сигнальных масок.

Опишите все что происходит в `main`: Инициализируем структуру `sigaction`, которая описывает поведения при обработке сигналов. Присваиваем обработчику `SIG_DFL`, который означает то, что устанавливается реакция по умолчанию.

Что значит по умолчанию? — Способом определенным системой.

sigemptyset: Сбрасывается маска множества сигналов, которые будут заблокированы (sa_mask).

sigfillset: Function initializes the set pointed to by set to include all signals defined by the system.).